

Prompt para IA de Desenvolvimento – Preparação Completa do Aplicativo para Publicação em Produção (Replit, Ambiente Local e Servidores Node.js)

Objetivo

Preparar integralmente o aplicativo para publicação em ambiente de produção, garantindo que funcione de forma idêntica e estável nos seguintes cenários:

- Replit (Deploy);
- Ambiente Local (Windows, Linux e macOS);
- Qualquer servidor com suporte a Node.js (cPanel Node.js, VPS, Cloud, Docker, Plesk, Railway, Render, DigitalOcean, AWS, Azure, Google Cloud, Hostinger, SuperDomínios e similares).

A aplicação deverá ser totalmente independente do ambiente em que foi desenvolvida, eliminando qualquer dependência específica do Replit ou de qualquer outra plataforma.

Ao final da implementação, o projeto deverá estar pronto para ser compilado, implantado e executado em qualquer ambiente compatível com Node.js sem necessidade de alterações no código-fonte.

Objetivos Gerais

Realizar uma auditoria completa do projeto e corrigir qualquer incompatibilidade relacionada a:

- ambiente de execução;
- sistema operacional;
- Node.js;
- npm;
- build;
- variáveis de ambiente;
- rotas;
- assets;
- APIs;
- SEO;
- PWA;
- Service Worker;
- streaming;
- segurança;
- desempenho;
- compatibilidade.

ETAPA 1 — Auditoria Geral do Projeto

Realizar uma inspeção completa em:

- estrutura de diretórios;
- dependências;
- scripts;
- arquivos de configuração;
- imports;
- assets;
- imagens;
- ícones;
- manifest;
- Service Worker;
- metatags;
- rotas;
- APIs;
- player;
- paginação;
- PWA.

Identificar automaticamente qualquer configuração dependente do ambiente original de desenvolvimento.

ETAPA 2 — Remover Dependências Específicas do Ambiente

Eliminar qualquer dependência exclusiva do Replit ou de qualquer outro ambiente.

Verificar:

- URLs específicas;
- caminhos absolutos;
- variáveis exclusivas;
- plugins;
- configurações internas;
- bibliotecas utilizadas apenas no ambiente de desenvolvimento.

Garantir que o projeto seja completamente portátil.

ETAPA 3 — Compatibilidade com Node.js

Garantir compatibilidade com versões LTS atuais do Node.js.

Validar:

- Node.js 20 LTS;
- Node.js 22 LTS (quando disponível).

Eliminar APIs obsoletas ou experimentais.

ETAPA 4 — Dependências

Executar auditoria completa em:

- `package.json`;
- `package-lock.json`;
- dependências diretas;
- dependências transitivas;
- dependências peer;
- dependências opcionais.

Atualizar apenas versões estáveis e compatíveis.

Remover dependências não utilizadas.

Eliminar conflitos de versões.

ETAPA 5 — Build de Produção

Garantir que:

- `npm install`
- `npm ci`
- `npm run build`
- `npm run preview` (quando aplicável)

executem sem erros.

Nenhum aviso crítico poderá permanecer.

ETAPA 6 — Estrutura da Pasta `dist`

Garantir que a pasta `dist` seja totalmente autossuficiente para publicação.

Verificar:

- assets;
- imagens;
- CSS;
- JavaScript;
- fontes;
- manifest;
- favicon;
- Open Graph Images;
- Service Worker.

Todos os caminhos devem permanecer válidos após o build.

ETAPA 7 — Assets

Organizar todos os recursos gráficos.

Estrutura recomendada:

```
dist/  
assets/  
images/  
icons/  
logos/  
og/  
favicons/  
fonts/  
css/  
js/
```

Atualizar automaticamente todas as referências.

ETAPA 8 — Variáveis de Ambiente

Verificar:

- `.env`
- `.env.production`
- `.env.local`

Garantir que nenhuma configuração dependa de:

- localhost;
- portas fixas;
- caminhos absolutos.

Utilizar apenas variáveis configuráveis.

ETAPA 9 — APIs

Validar:

- URLs;
- HTTPS;
- CORS;
- autenticação;
- timeout;
- reconexão;
- tratamento de erros.

Garantir funcionamento tanto no Replit quanto em qualquer servidor Node.js.

ETAPA 10 — Streaming

Validar:

- rádios provenientes da API;
- rádios provenientes de links externos;
- buffer;
- troca automática;
- reconexão;
- carregamento.

Confirmar funcionamento após o build.

ETAPA 11 — SEO

Auditar completamente:

- title;
- description;
- canonical;
- robots;
- sitemap;
- JSON-LD;
- Open Graph;
- Twitter Cards.

Confirmar funcionamento em ambiente publicado.

ETAPA 12 — Open Graph

Garantir que:

- imagens;
- título;
- descrição;

apareçam corretamente em:

- WhatsApp;
- Facebook;
- Instagram;
- LinkedIn;
- Telegram;
- Discord;
- X (Twitter);
- Google Chat.

Utilizar URLs absolutas.

ETAPA 13 — PWA

Validar:

- manifest;
- Service Worker;
- cache;
- ícones;
- splash screens;
- instalação.

Confirmar instalação correta em Android e Desktop.

ETAPA 14 — Performance

Executar auditoria utilizando Lighthouse.

Metas mínimas:

- Performance ≥ 95
 - Accessibility ≥ 95
 - Best Practices ≥ 95
 - SEO ≥ 100
-

ETAPA 15 — Segurança

Verificar:

- HTTPS;
- CSP (quando aplicável);
- Headers;
- MIME Types;
- CORS;
- Sanitização;
- Dependências vulneráveis.

Executar auditoria de segurança das dependências.

ETAPA 16 — Compatibilidade

Testar obrigatoriamente em:

Desktop:

- Chrome;
- Edge;
- Firefox;
- Brave;
- Opera.

Mobile:

- Android;
 - Chrome Mobile;
 - Samsung Internet;
 - Edge Mobile.
-

ETAPA 17 — Hospedagem

Validar funcionamento em:

- Replit Deploy;
- Ambiente Local;
- cPanel com Node.js;
- VPS Linux;
- Docker;
- Railway;
- Render;
- DigitalOcean;
- AWS;

- Azure;
- Google Cloud;
- Hostinger;
- SuperDomínios.

Nenhuma alteração manual deverá ser necessária para mudar de ambiente.

ETAPA 18 — Scripts

Garantir a existência e o funcionamento dos scripts necessários:

- desenvolvimento;
- build;
- preview;
- produção.

Os scripts devem ser compatíveis em qualquer sistema operacional.

ETAPA 19 — Logs

Remover:

- logs temporários;
- mensagens de depuração;
- código de teste;
- comentários desnecessários.

Manter apenas logs essenciais.

ETAPA 20 — Documentação

Gerar automaticamente documentação contendo:

- requisitos mínimos;
- versão do Node.js;
- versão do npm;
- instruções de instalação;
- instruções de build;
- instruções de deploy;
- variáveis de ambiente;
- estrutura do projeto;
- solução para problemas comuns.

Criar ou atualizar o arquivo `README.md` com todas essas informações.

ETAPA 21 — Testes Finais

Executar validação completa da aplicação publicada.

Confirmar:

- carregamento inicial;
- paginação;
- busca;
- categorias;
- favoritos;
- histórico;
- streaming;
- SEO;
- compartilhamento;
- PWA;
- Open Graph;
- imagens;
- APIs;
- assets;
- Service Worker;
- manifest.

Nenhum erro deverá aparecer:

- no Console;
- na aba Network;
- durante o Build;
- durante a execução em produção.

Critérios de Aceitação

O projeto somente será considerado concluído quando:

- funcionar corretamente no Replit;
- funcionar corretamente em ambiente local;
- funcionar corretamente em qualquer servidor Node.js;
- executar `npm install` sem erros;
- executar `npm ci` sem erros;
- executar `npm run build` sem erros;
- gerar uma pasta `dist` totalmente funcional;
- não depender de configurações específicas do ambiente de desenvolvimento;
- não apresentar erros de CORS, Assets, Rotas, APIs ou SEO;
- estar preparado para publicação imediata em produção.

Objetivo Final

Entregar uma aplicação totalmente pronta para produção, portátil, otimizada, segura e compatível com qualquer ambiente baseado em Node.js, garantindo que o mesmo código-fonte possa ser compilado, implantado e executado de forma idêntica no Replit, em ambiente local e em qualquer serviço de hospedagem compatível, sem necessidade de adaptações adicionais, preservando estabilidade, desempenho, SEO, PWA, streaming e experiência do usuário.

Prompt para IA de Desenvolvimento – Preparação Completa do Aplicativo para Publicação em Produção (Replit, Ambiente Local e Servidores Node.js)

Objetivo

Preparar o aplicativo para publicação definitiva em ambiente de produção, garantindo que funcione de forma idêntica e estável nos seguintes cenários:

- Replit (Deploy/Autoscale ou equivalente);
- Ambiente local executando a versão compilada (`npm run build`);
- Qualquer servidor com suporte ao Node.js (cPanel Node.js, VPS, Docker, PM2, Nginx, Apache, Cloud Run, Railway, Render, DigitalOcean, AWS, Azure, Google Cloud, entre outros).

O objetivo é eliminar qualquer dependência exclusiva do ambiente de desenvolvimento, do Replit ou do computador utilizado no desenvolvimento, tornando o projeto totalmente portátil, escalável e pronto para implantação em produção.

Objetivos Gerais

Realizar uma auditoria completa em toda a aplicação para garantir que:

- nenhuma funcionalidade dependa exclusivamente do ambiente de desenvolvimento;
 - nenhuma configuração dependa do Replit;
 - nenhuma funcionalidade dependa de caminhos absolutos do sistema operacional;
 - toda a aplicação funcione corretamente após a compilação (`npm run build`);
 - a aplicação possa ser publicada em qualquer servidor compatível com Node.js sem necessidade de adaptações adicionais.
-

ETAPA 1 — Auditoria Geral do Projeto

Realizar uma inspeção completa em:

- estrutura de diretórios;
- arquivos de configuração;
- dependências;
- scripts;
- assets;
- imagens;
- fontes;
- ícones;
- APIs;
- streaming;
- Service Worker;
- Manifest;
- SEO;
- Open Graph;
- Twitter Cards;
- rotas;
- paginação;
- carregamento das rádios;
- cache;
- PWA;
- variáveis de ambiente.

Corrigir automaticamente qualquer inconsistência encontrada.

ETAPA 2 — Compatibilidade entre Ambientes

Garantir funcionamento idêntico em:

Ambiente de Desenvolvimento

- `npm install`
- `npm run dev`

Ambiente de Produção Local

- `npm install`
- `npm run build`
- execução da pasta `dist`

Replit

- execução da aplicação;

- build;
- deploy;
- atualização automática;
- variáveis de ambiente;
- funcionamento em domínio próprio.

Servidores Node.js

Compatibilidade com:

- PM2;
 - Passenger;
 - Node.js do cPanel;
 - Docker;
 - Express;
 - Fastify;
 - Nginx;
 - Apache (Proxy Reverse);
 - Cloud Run;
 - Railway;
 - Render;
 - VPS Linux.
-

ETAPA 3 — Dependências

Auditar completamente:

- package.json;
- package-lock.json;
- versões do Node.js;
- versões do npm;
- dependências;
- dependências opcionais;
- peerDependencies.

Remover:

- dependências duplicadas;
- dependências não utilizadas;
- dependências exclusivas do Replit;
- bibliotecas obsoletas.

Atualizar automaticamente apenas quando não houver risco de incompatibilidade.

ETAPA 4 — Build de Produção

Executar obrigatoriamente:

- instalação limpa das dependências;
- limpeza da pasta `dist`;
- novo build completo;
- validação dos assets gerados;
- validação dos hashes;
- validação dos chunks;
- validação do code splitting;
- validação do tree shaking.

Corrigir qualquer erro de compilação.

ETAPA 5 — Variáveis de Ambiente

Padronizar todas as variáveis.

Garantir compatibilidade com:

- `.env`
- `.env.local`
- `.env.production`
- `process.env`
- `import.meta.env`

Nunca permitir:

- URLs fixas;
 - localhost em produção;
 - caminhos absolutos;
 - credenciais embutidas no código.
-

ETAPA 6 — Assets

Verificar:

- imagens;
- favicons;
- Open Graph Images;
- Twitter Images;
- Manifest;
- ícones;
- splash screens;
- fontes.

Garantir que todos os assets sejam gerados corretamente após o build.

ETAPA 7 — SEO

Validar completamente:

- Title;
- Description;
- Keywords;
- Canonical;
- Robots;
- Sitemap;
- JSON-LD;
- Open Graph;
- Twitter Cards;
- Meta Robots;
- Language;
- Locale.

Confirmar que todas as imagens de compartilhamento funcionem corretamente.

ETAPA 8 — APIs

Validar:

- URLs;
- HTTPS;
- CORS;
- autenticação;
- disponibilidade;
- tratamento de erros;
- timeout;
- reconexão.

Garantir funcionamento tanto no Replit quanto em qualquer servidor Node.js.

ETAPA 9 — Streaming

Validar:

- rádios provenientes da API;
- rádios provenientes de links externos;
- carregamento do player;

- buffer;
- reconexão;
- troca automática;
- timeout.

Confirmar funcionamento após o build.

ETAPA 10 — Paginação

Garantir:

- funcionamento da primeira à última página;
 - carregamento de todas as rádios;
 - integração entre rádios locais e rádios da API;
 - atualização automática após filtros;
 - ausência de páginas vazias.
-

ETAPA 11 — PWA

Validar:

- Manifest;
 - Service Worker;
 - Offline Cache;
 - Atualizações;
 - Instalação;
 - Ícones;
 - Splash Screen;
 - Atalhos.
-

ETAPA 12 — Logs

Remover:

- logs de depuração;
- mensagens temporárias;
- código experimental;
- comentários obsoletos.

Manter apenas logs essenciais para diagnóstico em produção.

ETAPA 13 — Performance

Executar auditoria completa utilizando Lighthouse.

Objetivos mínimos:

- Performance ≥ 95
- Accessibility ≥ 95
- Best Practices ≥ 95
- SEO ≥ 100

Otimizar automaticamente sempre que possível.

ETAPA 14 — Segurança

Verificar:

- Content Security Policy;
- HTTPS;
- Headers;
- CORS;
- XSS;
- Clickjacking;
- MIME Types;
- Cache-Control.

Aplicar boas práticas sem comprometer o funcionamento do aplicativo.

ETAPA 15 — Compatibilidade

Testar obrigatoriamente em:

- Chrome;
- Edge;
- Firefox;
- Brave;
- Opera;
- Android;
- iOS.

Validar funcionamento em:

- Desktop;
 - Tablet;
 - Smartphone.
-

ETAPA 16 — Deploy

Preparar automaticamente o projeto para publicação em:

Replit

- Build;
- Deploy;
- Domínio próprio;
- SSL;
- Variáveis de ambiente.

Ambiente Local

- Build completo;
- Execução da pasta `dist`;
- Testes locais.

Servidores Node.js

Compatibilidade com:

- cPanel Node.js;
- PM2;
- Passenger;
- Docker;
- VPS;
- Reverse Proxy (Nginx e Apache).

Caso necessário, criar automaticamente:

- scripts de inicialização;
- scripts de build;
- scripts de deploy;
- scripts de restart.

ETAPA 17 — Documentação

Gerar documentação técnica contendo:

- requisitos mínimos;
- versão do Node.js recomendada;
- versão do npm recomendada;
- comandos de instalação;
- comandos de build;
- comandos de produção;
- comandos de atualização;

- estrutura de diretórios;
 - variáveis de ambiente;
 - instruções para hospedagem em Replit;
 - instruções para hospedagem em servidores Node.js;
 - instruções para hospedagem em cPanel.
-

ETAPA 18 — Validação Final

Somente considerar a implementação concluída quando for confirmado que:

- o projeto funciona em ambiente local;
- funciona no Replit;
- funciona em qualquer servidor Node.js;
- todas as rádios da API aparecem corretamente;
- todas as rádios externas aparecem corretamente;
- toda a paginação funciona;
- o streaming funciona corretamente;
- o SEO está totalmente validado;
- o Open Graph funciona corretamente;
- o Twitter Cards funciona corretamente;
- todas as imagens são carregadas;
- não existem erros de build;
- não existem erros de console;
- não existem erros de Network;
- não existem dependências exclusivas do Replit;
- não existem dependências do ambiente de desenvolvimento;
- a aplicação está pronta para publicação imediata em produção.

Objetivo Final

Entregar uma aplicação totalmente pronta para produção, portátil, otimizada, segura e independente do ambiente de desenvolvimento, funcionando de forma idêntica no Replit, em ambiente local após o `npm run build` e em qualquer servidor com suporte a Node.js, incluindo hospedagens compartilhadas com cPanel, VPS, serviços em nuvem e plataformas de deploy modernas. A solução deverá seguir as melhores práticas de desenvolvimento, desempenho, SEO, segurança, PWA e manutenção, garantindo estabilidade, escalabilidade e facilidade de implantação em qualquer infraestrutura compatível.